PROYECTO: BAGELS

Instituto Maobits

Arquitectura Mental y Tecnológica para Negocios

www.institutomaobits.com

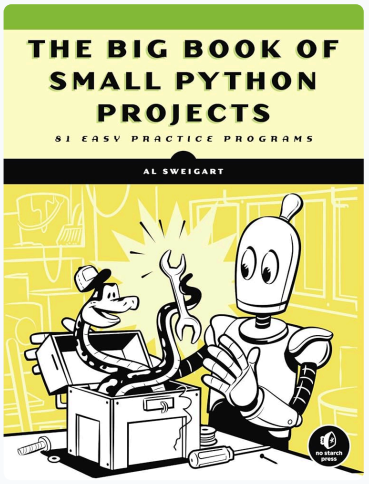
Fundador: Mauricio Chara Hurtado

Empresario del sector de las ciencias de la computación

Perfil Profesional: Empresario informático con más de 10 años de experiencia en el desarrollo de soluciones tecnológicas, formación digital y transformación de procesos mediante pensamiento estructurado, modelamiento y arquitectura lógica.

Lidero el modelo educativo Maobits, enfocando la arquitectura mental y tecnológica aplicada a los negocios. Integrando desarrollo de software, producción audiovisual y marketing digital para la transformación empresarial y educativa.

Créditos de Obra Base



Basado en el proyecto: **“Bagels”**

Libro: "The Big Book of Small Python Projects"

Autor: **Al Sweigart**

Licencia: **Creative Commons**

Este proyecto se utiliza como modelo de arquitectura mental aplicada, formando parte de una colección curada de programas simples en Python diseñados para aprender haciendo.

Descripción del Proyecto

Este programa implementa "Bagels";, un clásico juego de lógica deductiva. El objetivo del usuario es adivinar un número secreto de N dígitos (sin números repetidos) basándose en pistas de inferencia lógica proporcionadas por el sistema después de cada intento.

Funcionalidades Clave:

- Generación automática de un número secreto de longitud configurable.
- Sistema de pistas lógicas para guiar la deducción:
  - **Pico:** Un dígito es correcto, pero está en la posición incorrecta.
  - **Fermi:** Un dígito es correcto y está en la posición correcta.
  - **Bagels:** Ningún dígito es correcto.
- Control de intentos máximos y opción para jugar múltiples partidas consecutivas.

Conceptos de Programación Aplicados:

- Generación de números aleatorios y manipulación de secuencias.
- Bucles de control de flujo (`while`, `for`).
- Validación estricta de entrada de datos del usuario.
- Lógica condicional avanzada (`if`, `elif`, `else`).
- Manejo de cadenas de texto (strings) y listas.

Explicación Paso a Paso del Código

A continuación, desglosaremos el código fuente del juego Bagels para entender cómo funciona cada una de sus partes, desde la configuración inicial hasta el motor de inferencia lógica.

1. Importación de Librerías

El programa comienza importando el módulo estándar necesario para la aleatoriedad.

```
# Importamos la librería random para generar números aleatorios.
import random
```

`random`: Es fundamental para este juego, ya que nos permite mezclar los dígitos posibles para generar un número secreto impredecible en cada partida.

2. Configuración del Sistema (Constantes)

Se definen variables en mayúsculas, lo que en programación indica que son constantes (valores que configuran el comportamiento global del sistema y no cambian durante la ejecución).

```
# Número de dígitos que tendrá el número secreto.
NUM_DIGITOS = 3

# Número máximo de intentos permitidos.
MAX_INTENTOS = 10
```

`NUM_DIGITOS` define la longitud del número a adivinar (por defecto 3). `MAX_INTENTOS` establece la dificultad limitando cuántas oportunidades tiene el jugador. Al usar constantes, modificar la dificultad del juego requiere cambiar solo estas dos líneas.

3. Función Principal (Controlador del Sistema)

La función `main()` es el cerebro del programa. Controla el flujo general, las instrucciones y los bucles de las partidas.

```
def main():
    print(f""" Bagels – Juego de Lógica Deductiva
    ... [Instrucciones del juego] ...
    """)

    while True:
        numero_secreto = obtener_numero_secreto()
        intentos = 1

        while intentos <= MAX_INTENTOS:
            intento = ""
            # Validación de entrada
            while len(intento) != NUM_DIGITOS or not intento.isdecimal
                print(f"Intento #{intentos}:")
                intento = input("> ")

            # Obtener pistas y verificar victoria
            pistas = obtener_pistas(intento, numero_secreto)
            print(pistas)

            if intento == numero_secreto:
```

```
                break

            intentos += 1

            if intentos > MAX_INTENTOS:
                print("Se acabaron los intentos.")
                print(f"El número secreto era {numero_secreto}.")

        print("¿Quieres jugar otra vez? (si/no)")
        if not input("> ").lower().startswith("s"):
            break
```

¿Cómo funciona?

- Utiliza un bucle `while True` externo para permitir jugar múltiples veces sin reiniciar el programa.
- El bucle interno `while intentos <= MAX_INTENTOS` controla el desarrollo de una partida individual.
- **Validación robusta:** `len(intento) != NUM_DIGITOS or not intento.isdecimal()` asegura que el programa no avance ni gaste un intento si el usuario ingresa letras o una cantidad incorrecta de números.

4. Función: Generar Número Secreto

Esta función se encarga de crear el objetivo del juego: un número aleatorio sin dígitos repetidos.

```
def obtener_numero_secreto():
    # Crear lista de dígitos del 0 al 9
    numeros = list("0123456789")

    # Mezclar aleatoriamente los dígitos
    random.shuffle(numeros)

    # Construir número secreto
    numero_secreto = ""
    for i in range(NUM_DIGITOS):
        numero_secreto += numeros[i]

    return numero_secreto
```

En lugar de generar un número aleatorio tradicional (que podría tener dígitos repetidos como "112"), el programa crea una lista con los caracteres del '0' al '9'. Luego, usa `random.shuffle()` para desordenar la lista y concatena los primeros `NUM_DIGITOS` para formar el número secreto perfecto.

5. Función: Motor de Inferencia (Pistas)

Esta es la parte lógica central del proyecto. Compara la entrada del usuario con el número secreto y deduce qué pistas entregar.

```
def obtener_pistas(intento, numero_secreto):
    if intento == numero_secreto:
        return "¡Correcto! Has adivinado el número."

    pistas = []
    for i in range(NUM_DIGITOS):
        if intento[i] == numero_secreto[i]:
            pistas.append("Fermi")
        elif intento[i] in numero_secreto:
            pistas.append("Pico")

    if not pistas:
        return "Bagels"

    pistas.sort()
    return " ".join(pistas)
```

Lógica de evaluación:

- Itera sobre cada posición del intento usando `range(NUM_DIGITOS)`.
- Si el dígito coincide en valor y posición (`intento[i] == numero_secreto[i]`), agrega "Fermi".
- Si el dígito existe en el número secreto pero en otra posición (`intento[i] in numero_secreto`), agrega "Pico".
- Si la lista de pistas está vacía al final del bucle, retorna "Bagels".
- **Optimización inteligente:** `pistas.sort()` ordena alfabéticamente las pistas antes de entregarlas. Esto es crucial; si no se ordenaran, el jugador sabría a qué dígito exacto corresponde cada pista basándose en el orden de las palabras, arruinando el desafío deductivo.

6. Ejecución del Programa

Finalmente, el script incluye la condición estándar de Python para ejecutar la función principal.

```
if __name__ == "__main__":
    main()
```

Esta estructura asegura que `main()` solo se ejecute si el archivo se corre directamente. Si este código fuera importado como un módulo en otro proyecto más grande, el juego no se iniciaría automáticamente, lo cual es una excelente práctica de arquitectura de software.

Reference

[1] https://inventwithpython.com/bigbookpython/
https://inventwithpython.com/bigbookpython/